

```

        Thread worker = new Thread(new ThreadStart(
            delegate()
            {
                // Simulate some gnarly work to
do by a boring loop
                for (int i = 0; i < 1000000000;
i++) { }

                MessageBox.Show("Done!");
            }));
        worker.Start();
    });
    this.Controls.Add(button);
    this.Controls.Add(input);
}

public static void Test()
{
    Application.Run(new ConcurrentForm());
}

```

You can think of a thread as a sequence of instructions that can operate in parallel with other threads. Notice how we did not have to do anything more once we started the thread. This is because the runtime/operating system is responsible for scheduling when threads run. These context switches between threads occur quickly enough that even on a single-processor machine, we maintain the illusion that different threads operate in parallel.

Threads maintain their own copies of local variables, but objects stored on the heap (e.g., reference objects) are shared.

```

public class BasicThreadExample
{
    public static void Test()
    {
        Thread t1 = new Thread(new ThreadStart(PrintSeq));
        t1.Name = "t1";
        Thread t2 = new Thread(new ThreadStart(PrintSeq));
        t2.Name = "t2";
        t1.Start();
        t2.Start();
    }
}

```